

HOSPITAL MANAGEMENT SYSTEM

Java Programming Project

Submitted as part of the VITyarthi Build Your Own Project

Student Name: Jenil Solanki

Course: Java Programming

Branch: B.Tech Computer Science and Engineering

University: VIT Bhopal University

1. Introduction

Hospitals handle a large amount of information every day, including patient details, doctor information, appointments, medical records, room allocation, billing, and staff information. Managing all these details manually can become difficult and may result in errors or confusion.

The Hospital Management System is a Java-based console application developed to manage these basic hospital activities in an organized way.

The system provides different menu-based modules for managing patients, doctors, appointments, medical records, rooms, bills, and staff. Users can enter information through the console and perform different operations according to their requirements.

The project is developed completely using Java and its standard libraries. It focuses on applying concepts learned in Java Programming such as classes and objects, methods, ArrayList, loops, conditional statements, exception handling, input validation, and date handling.

The main purpose of the project is to create a simple and functional system that demonstrates how Java programming concepts can be applied to solve a real-world problem.

2. Problem Statement

Managing hospital information manually can become difficult when the number of patients, doctors, appointments, rooms, and bills increases.

Important information may be difficult to search or update quickly. Appointment scheduling can also create problems if the same doctor is booked more than once on the same date. Similarly, calculating different charges manually can lead to mistakes.

Therefore, there is a need for a simple system that can organize basic hospital information and provide easy operations for adding, searching, updating, viewing, and managing records.

The proposed Hospital Management System solves this problem through a menu-driven Java application.

3. Objectives

The main objectives of the project are:

1. To develop a simple hospital management system using Java.
 2. To manage patient information in an organized way.
 3. To manage doctor information and availability.
 4. To provide appointment booking and management.
 5. To maintain basic medical records.
 6. To manage hospital room allocation.
 7. To calculate and display patient bills.
 8. To manage basic staff information.
 9. To validate user inputs and handle incorrect inputs.
 10. To apply Java programming concepts to a practical real-world problem.
-

4. Scope of the Project

The project focuses on the basic management activities of a hospital.

The system currently includes:

- Patient management
- Doctor management
- Appointment management
- Medical record management
- Room management
- Billing
- Staff management
- Input validation
- Error handling

The project is designed as a console-based application. It does not currently include a graphical user interface or a database.

The current data is maintained during program execution using Java collections.

5. Target Users

The system can be used as a basic management application by:

- Hospital reception staff
- Hospital administrators
- Nurses or support staff
- Students learning Java programming
- Small healthcare facilities for basic record management

The current version is mainly intended as an educational Java project demonstrating the implementation of a real-world management system.

6. Functional Requirements

6.1 Patient Management

The system should allow the user to:

- Add a new patient
- View all patients
- Search for a patient
- Update patient information
- Delete a patient

Patient information includes:

- Patient ID
 - Name
 - Age
 - Gender
 - Phone number
 - Address
-

6.2 Doctor Management

The system should allow the user to:

- Add a doctor
- View all doctors
- Search for doctors

Doctor information includes:

- Doctor ID
 - Doctor name
 - Specialization
 - Phone number
 - Availability
 - Consultation fee
-

6.3 Appointment Management

The appointment module allows the user to:

- Book an appointment
- View appointments
- Cancel an appointment
- Complete an appointment

Each appointment contains:

- Appointment ID
- Patient ID
- Doctor ID
- Appointment date
- Appointment status

The system also checks whether the same doctor has already been booked for the selected date.

6.4 Medical Records

The system allows users to:

- Add medical records
- View all medical records
- View records belonging to a particular patient

Medical records contain:

- Record ID
- Patient ID
- Doctor ID
- Diagnosis
- Prescription
- Remarks

6.5 Room Management

The room module provides:

- Add room
- View rooms
- Assign room
- Release room

Room information includes:

- Room number
- Room type
- Daily charge
- Room status

Room types can include General, Semi-Private, Private, and ICU.

6.6 Billing

The billing module allows users to:

- Generate a bill
- View all bills
- Search for a bill

The bill calculates:

- Consultation charge
- Room charge
- Medicine charge
- Test charge

- Total amount

The total amount is calculated automatically.

Total = Consultation Charge + Room Charge + Medicine Charge + Test Charge

6.7 Staff Management

The staff module allows users to:

- Add staff
- View staff information

Staff information includes:

- Staff ID
 - Name
 - Role
 - Department
 - Phone number
-

7. Non-Functional Requirements

The project also considers the following non-functional requirements, in line with the VITyarthi project requirements.

7.1 Usability

The application uses a simple menu-driven interface so that users can select the required operation easily.

7.2 Reliability

The system performs validation checks before accepting important information and handles invalid inputs without immediately terminating the program.

7.3 Error Handling

Invalid numerical values, incorrect phone numbers, invalid dates, and invalid menu choices are handled using validation and exception handling.

7.4 Maintainability

Different operations are organized into separate methods and different entities are represented using separate classes, making the program easier to understand and modify.

7.5 Performance

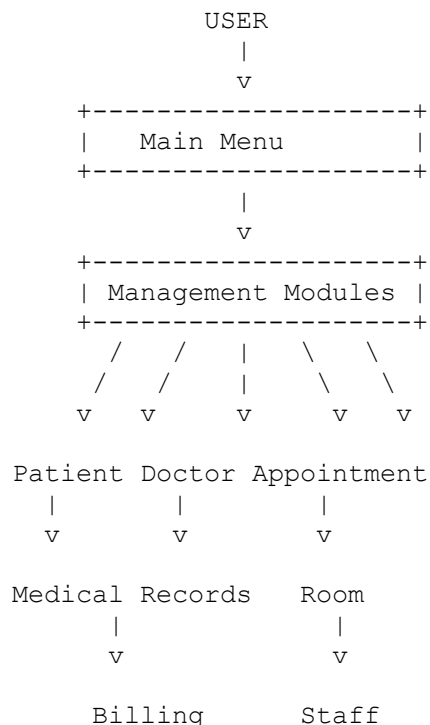
The system uses Java ArrayList collections for storing records during program execution. For the intended small-scale project, searching and managing records is straightforward.

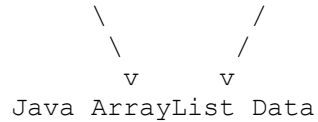
7.6 Resource Efficiency

The application is console-based and uses only Java standard libraries, so it does not require additional frameworks or large external resources.

8. System Architecture

The project follows a simple layered flow:





The user interacts with the main menu. Based on the selected option, the program calls the corresponding management module.

The information is stored using Java ArrayList collections while the application is running.

9. System Workflow

The basic workflow of the application is:

```
Start
|
v
Display Main Menu
|
v
Select Module
|
+----> Patient Management
|
+----> Doctor Management
|
+----> Appointment Management
|
+----> Medical Records
|
+----> Room Management
|
+----> Billing
|
+----> Staff Management
|
v
Perform Selected Operation
|
v
Validate Input
|
v
Display Result
|
v
Return to Main Menu
|
+----> Continue
|
+----> Exit
```

|
v
End

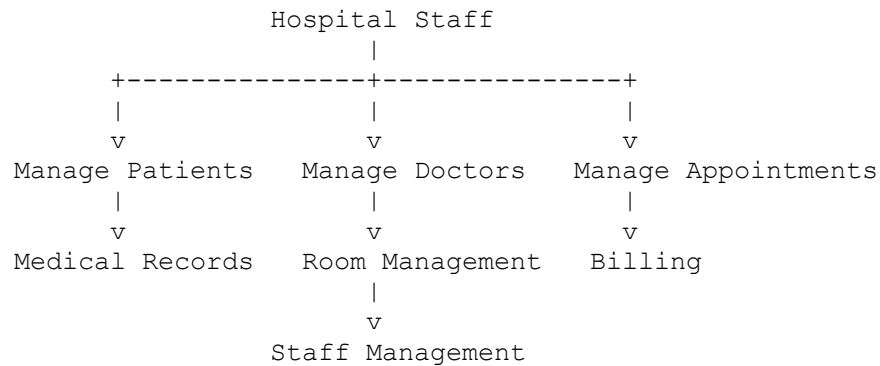
10. Design Diagrams

10.1 Use Case Diagram

Main Actor

Hospital Staff / System User

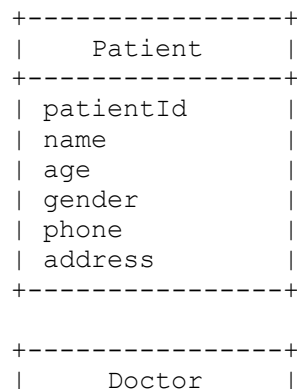
Use Cases



The main user interacts with all the management modules through the application menu.

11. Class Design

The project uses multiple Java classes to represent different entities.



+-----+		
	doctorId	
	name	
	specialization	
	phone	
	availability	
	fee	
+-----+		

+-----+		
	Appointment	
+-----+		
	appointmentId	
	patientId	
	doctorId	
	date	
	status	
+-----+		

+-----+		
	MedicalRecord	
+-----+		
	recordId	
	patientId	
	doctorId	
	diagnosis	
	prescription	
	remarks	
+-----+		

+-----+		
	Room	
+-----+		
	roomNumber	
	roomType	
	charge	
	status	
+-----+		

+-----+		
	Bill	
+-----+		
	billId	
	patientId	
	consultation	
	roomCharge	
	medicineCharge	
	testCharge	
	total	
+-----+		

+-----+		
	Staff	
+-----+		
	staffId	
	name	
	role	

```
| department |  
| phone     |  
+-----+
```

12. Sequence Flow for Appointment Booking

A simplified appointment booking process is:

```
User  
|  
| Select Appointment Management  
v  
System  
|  
| Select Book Appointment  
v  
System asks for Appointment ID  
|  
v  
System asks for Patient ID  
|  
v  
System asks for Doctor ID  
|  
v  
System asks for Appointment Date  
|  
v  
System validates information  
|  
+---- Invalid -----> Display Error  
|  
+---- Valid  
|  
v  
Check Doctor Booking  
|  
+---- Already booked  
|      |  
|      v  
|   Display Error  
|  
+---- Available  
|  
v  
    Create Appointment  
|  
v  
    Display Success Message
```

13. Design Decisions and Rationale

13.1 Console-Based Interface

A console interface was selected because the project focuses on Java programming concepts rather than GUI development.

It also keeps the project simple and allows the main functionality to be implemented using core Java.

13.2 ArrayList

ArrayList was used to store patients, doctors, appointments, records, rooms, bills, and staff.

It provides a simple way to store multiple objects and add or remove records during program execution.

13.3 Separate Classes

Separate classes were created for important entities such as Patient, Doctor, Appointment, MedicalRecord, Room, Bill, and Staff.

This makes the relationship between the real-world hospital entities and Java objects easier to understand.

13.4 Menu-Based Navigation

A main menu and individual submenus were used so the user can easily select the required operation.

13.5 Input Validation

Validation was included to prevent common input errors such as invalid age, incorrect phone numbers, negative charges, invalid dates, and invalid menu choices.

13.6 Java Standard Library

Only standard Java libraries are used. This keeps the project easy to compile and run without requiring additional software or external dependencies.

14. Implementation Details

The project is implemented using Java.

The main class is:

```
HospitalManagementSystem
```

The supporting classes are:

```
Patient  
Doctor  
Appointment  
MedicalRecord  
Room  
Bill  
Staff
```

The program uses:

```
ArrayList
```

to store objects.

For user input:

```
Scanner
```

is used.

For appointment dates:

```
LocalDate  
DateTimeFormatter
```

are used.

Exception handling is implemented for invalid input, especially when the user enters an incorrect data type.

The program uses methods to divide different operations into manageable sections such as adding patients, displaying doctors, booking appointments, generating bills, and managing rooms.

15. Input Validation

Several validation checks are included in the application.

Age

The system accepts a valid age within the specified range.

Phone Number

The phone number is checked to ensure that it contains 10 digits.

Charges

Negative values are not accepted for consultation, room, medicine, or test charges.

Appointment Date

The date must follow the required format:

dd-MM-yyyy

The system also checks the appointment date before accepting it.

Duplicate IDs

The system checks IDs before adding records where required.

Doctor Appointment

The system checks whether a doctor has already been booked for the selected date.

16. Screenshots and Results

The following screenshots should be added to the final project report after running the program.

Screenshot 1 — Main Menu

Show the main Hospital Management System menu.

Caption: Main menu displaying all available hospital management modules.

Screenshot 2 — Patient Management

Show a patient being added and displayed.

Caption: Patient management module showing successful patient registration.

Screenshot 3 — Doctor Management

Show doctor details.

Caption: Doctor management module displaying doctor information and consultation fee.

Screenshot 4 — Appointment Management

Show a successfully booked appointment.

Caption: Appointment module showing a successfully created appointment.

Screenshot 5 — Medical Records

Show a medical record.

Caption: Medical records module displaying diagnosis, prescription, and remarks.

Screenshot 6 — Room Management

Show room availability or assignment.

Caption: Room management module showing room status.

Screenshot 7 — Billing

Show the generated bill and total amount.

Caption: Billing module calculating the total amount from individual hospital charges.

Screenshot 8 — Staff Management

Show staff information.

Caption: Staff management module displaying hospital staff details.

17. Testing Approach

The application was tested using different normal and invalid inputs.

Test Case 1 — Add Patient

Input: Valid patient information.

Expected Result: Patient should be added successfully.

Actual Result: Patient was added and displayed correctly.

Status: PASS

Test Case 2 — Search Patient

Input: Existing patient ID.

Expected Result: Patient information should be displayed.

Actual Result: Correct patient information was displayed.

Status: PASS

Test Case 3 — Book Appointment

Input: Valid patient ID, doctor ID, and appointment date.

Expected Result: Appointment should be created.

Actual Result: Appointment was created successfully.

Status: PASS

Test Case 4 — Duplicate Appointment

Input: Existing appointment ID.

Expected Result: Duplicate appointment should not be created.

Actual Result: System rejected the duplicate ID.

Status: PASS

Test Case 5 — Same Doctor on Same Date

Input: Doctor already booked on the selected date.

Expected Result: Second appointment should be rejected.

Actual Result: System prevented the duplicate doctor booking.

Status: PASS

Test Case 6 — Room Assignment

Input: Available room number.

Expected Result: Room should become occupied.

Actual Result: Room status was updated correctly.

Status: PASS

Test Case 7 — Bill Calculation

Input:

Consultation	=	₹500
Room	=	₹1000
Medicine	=	₹200
Tests	=	₹300

Expected Result:

Total = ₹2000

Actual Result:

Total = ₹2000

Status: PASS

Test Case 8 — Invalid Input

Input: Invalid menu choice or incorrect data type.

Expected Result: Program should handle the input and ask the user to enter valid information.

Actual Result: Invalid input was handled without terminating the program.

Status: PASS

18. Testing Summary

The complete application was compiled and tested.

The major modules were tested using valid and invalid inputs.

The following areas were successfully verified:

Module	Result
Patient Management	PASS
Doctor Management	PASS
Appointment Management	PASS
Medical Records	PASS
Room Management	PASS
Billing	PASS
Staff Management	PASS
Input Validation	PASS
Error Handling	PASS
Program Exit	PASS

19. Challenges Faced

During development, some challenges were faced while connecting the different modules.

One challenge was maintaining different types of information such as patients, doctors, appointments, and bills without using a database.

Another challenge was validating user input. Since the program takes information directly from the console, incorrect values could cause problems if they were not handled properly.

Appointment management also required additional checking so that the same doctor could not be booked for two appointments on the same date.

Calculating the total bill was another part where the different charges had to be combined correctly.

Handling these problems helped in understanding how different Java concepts can work together in a single project.

20. Learnings and Key Takeaways

This project helped in understanding the practical use of Java programming concepts.

The main learnings from the project are:

- Creating and using Java classes and objects
- Using ArrayList for storing multiple objects
- Creating methods for different operations
- Using loops and conditional statements
- Taking input using Scanner
- Handling exceptions
- Validating user input
- Working with dates using LocalDate
- Organizing a larger Java program
- Connecting different modules through a menu-driven workflow
- Testing a program using different input cases

The project also helped in understanding how programming concepts can be applied to a real-world management problem.

21. Future Enhancements

The current version is a console-based application. It can be improved in the future by adding:

1. Database connectivity for permanent data storage.
2. Login and authentication for different users.
3. A graphical user interface.
4. Medicine inventory management.
5. Laboratory management.
6. Patient discharge management.
7. Automatic bill printing.
8. Backup and recovery functionality.
9. More detailed doctor availability management.
10. Improved reporting and searching features.

These improvements can make the system more suitable for larger-scale use.

22. Conclusion

The Hospital Management System successfully demonstrates how Java programming can be used to solve a practical management problem.

The application provides different modules for managing patients, doctors, appointments, medical records, rooms, billing, and staff.

The project uses core Java concepts such as classes, objects, methods, `ArrayList`, loops, conditional statements, exception handling, input validation, and date handling.

Testing showed that the main functions work correctly with both valid and invalid inputs.

Although the current project is a console-based application without permanent database storage, it provides a basic foundation that can be expanded with additional features in the future.

Overall, the project demonstrates the practical application of Java programming concepts in building a functional management system.

23. References

1. Oracle Java Documentation — Java Standard Library and Java Programming Concepts.
2. VITyarthi — Build Your Own Project guidelines.
3. Java documentation for `ArrayList`, `Scanner`, `LocalDate`, and exception handling.
4. Course material provided for the Java Programming course.

5. VIT Bhopal University academic resources.
-

24. Project Information

Project Title: Hospital Management System

Programming Language: Java

Application Type: Console-Based Application

Main Concepts Used:

- Classes and Objects
- Methods
- ArrayList
- Encapsulation
- Loops
- Conditional Statements
- Exception Handling
- Input Validation
- Date Handling

Main Modules:

1. Patient Management
2. Doctor Management
3. Appointment Management
4. Medical Records
5. Room Management
6. Billing
7. Staff Management